

SIT720 Task 8.1D: Sydney housing prices

Machine learning mini project | 17 September 2026

1. Problem and data collection

This project predicts a property sale price in Australian dollars. Parramatta, Blacktown and Mosman were selected and confirmed as contrasting buying options. The collected Parramatta sample is mainly apartments, Blacktown mixes houses and apartments, and Mosman includes much more expensive homes. Location, accommodation and property type may therefore affect prices.

The dataset contains 119 sold properties: 37 Parramatta, 43 Blacktown and 39 Mosman. Listing facts were manually transcribed by the AI assistant from Domain and realestate.com.au on 16 September 2026. Every row has a source URL. This is not a claim that the student personally collected or independently verified the sales.

Recorded fields	Use
Sale price (AUD)	Prediction target
Suburb, type, bedrooms, bathrooms, parking, sale date	Model inputs
Address, advertised area, source, access date, property ID	Checking and traceability; excluded from prediction

Collection problems included withheld prices, missing parking, inconsistent area definitions and changing listing details. Withheld-price and retirement listings were excluded. Missing parking stays unknown. Advertised area is retained for checking but excluded because it mixes land, floor and whole-building area. All 119 transcriptions were checked against the viewed listing text; this does not independently verify settlement records.

The sample is convenient rather than random. Disclosed prices may be selective, several units share buildings, and the Mosman house search was supplemented separately. Results should not be presented as official suburb statistics or as reliable estimates for all Sydney properties.

Submission details

Application: [Open the deployed housing predictor \(owner-private\)](#)

Dataset and source archive: **Accessible archive URL not yet supplied.** Upload the included ZIP to OneDrive/Dropbox or GitHub and add its link in `submission_links.json`, then rebuild this report.

Part 5 limitation: genuine personal estimates were not supplied. Requested invented values are labelled AI illustrations. This report does not claim a completed human-performance comparison. These two items need attention before submission.

2. Understanding and preparing the data

A fixed split reserves 24 test properties: ten preselected comparison cases and fourteen additional cases sampled by suburb with seed 42. The remaining 95 support exploration and model selection. The test is not purely random, and nearby units can cross the split. Building-grouped and forward-time tests would be stronger.

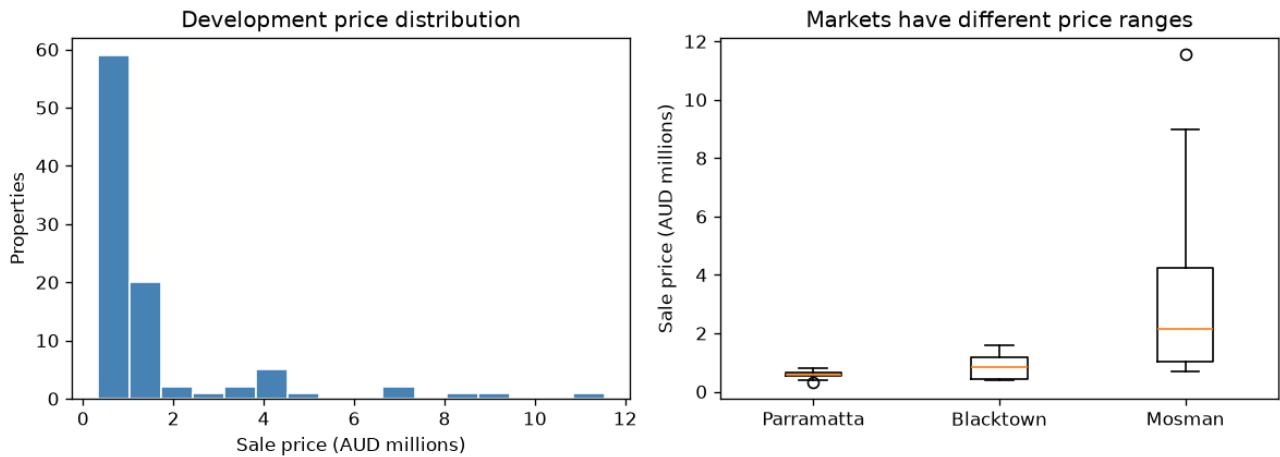


Figure 1. Development data only. The right tail contains expensive Mosman houses.

Development median prices are \$572,000 in Parramatta, \$854,000 in Blacktown and \$2,152,500 in Mosman. Prices are right-skewed. Expensive genuine sales are retained; removing them would conceal important failures.

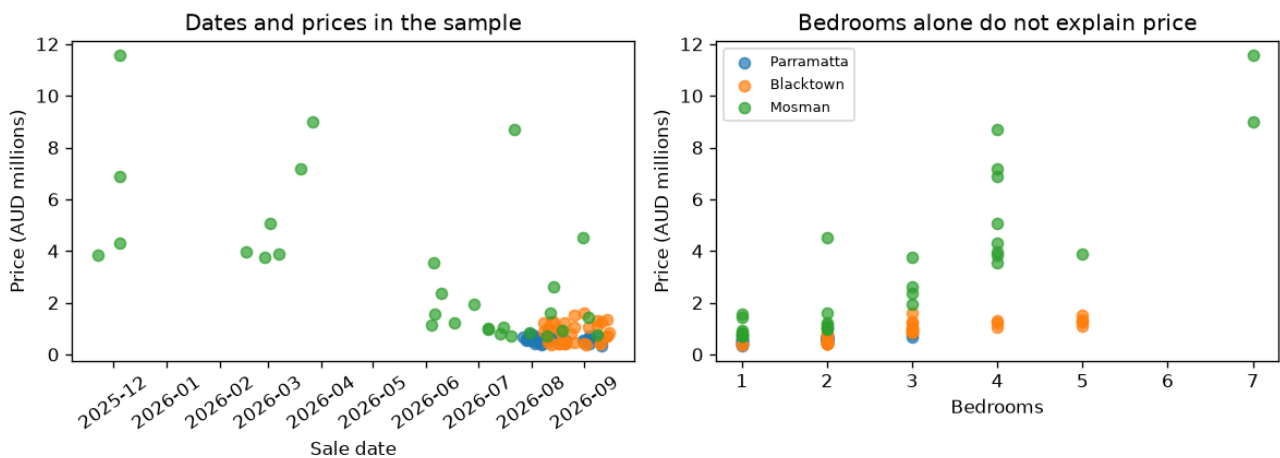


Figure 2. Sale dates and bedrooms. Older observations are mainly Mosman homes.

The date plot cannot establish a market trend because suburb and property mix change over time. Before engineering, the expected three strongest inputs were suburb, property type and bedrooms: market location, dwelling form and accommodation. Month index and bathrooms per bedroom were then added without using prices; studios use a denominator of one.

Within each cross-validation fold, missing numeric inputs use the training median, numeric features are standardised, and categories are one-hot encoded. This prevents preprocessing leakage. Scaling is essential for distance-based KNN. Address and listing identifiers are excluded.

3. Model development and evaluation

KNN averages similar sales, a decision tree learns rules, and random forest averages many trees. Before training, forest was expected to win because averaging can reduce tree variance and capture interactions. Five shuffled folds compare a small parameter grid using MAE. RMSE highlights large errors; R-squared compares squared error with a mean-price baseline.

Model	CV MAE	CV RMSE	CV R ²	Train MAE
KNN	\$437,074	\$888,508	0.625	\$317,782
Random forest	\$454,019	\$1,006,365	0.068	\$303,296
Decision tree	\$581,342	\$1,311,502	-5.053	\$376,494

Table 1. Mean fold scores at selected settings: KNN k=3; tree unrestricted depth/minimum leaf 3; forest 200 trees, unrestricted depth, minimum leaf 2 and max_features=0.8. KNN k=3/5/9, tree depth 2/4/unrestricted and forest depth 4/unrestricted were tested.

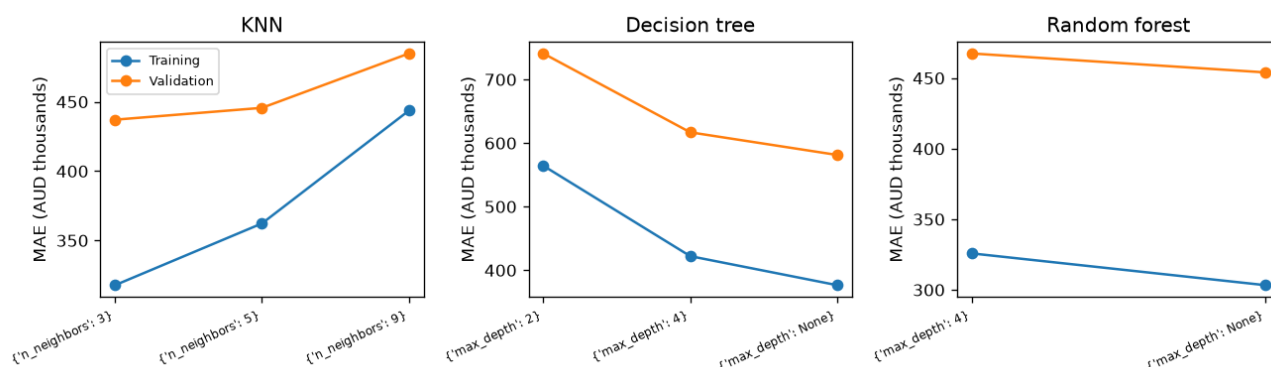


Figure 3. Training and validation MAE at each tested model complexity.

KNN is selected by CV MAE (\$437,074), against the initial forest expectation. Its \$16,946 advantage is smaller than fold variability (KNN SD \$207,485; forest \$156,428). Higher k increases both errors, suggesting excessive smoothing. Deeper trees improve both scores here, although training-validation gaps remain. More complexity is not automatically worse. Mean fold R-squared is unstable when small folds have different price variance.

24-property test	MAE	RMSE	R ²
KNN	\$1,202,750	\$3,731,553	0.384
Decision tree	\$1,208,407	\$3,509,637	0.455
Random forest	\$1,202,161	\$3,593,521	0.429
Median baseline	\$2,228,000	\$5,191,548	-0.193

KNN improves on the median baseline but test errors are large. Forest has slightly lower test MAE and the tree lower RMSE; the CV choice is retained to avoid selecting on test results. CV scores also served tuning, so they are mildly optimistic. KNN is recommended only for this study prototype.

4. Investigating the five largest errors

Property	Actual	KNN estimate	Absolute error
17 Morella Road	\$23,000,000	\$5,391,000	\$17,609,000
10 Cyprian Street	\$8,500,000	\$5,391,000	\$3,109,000
53 - 53A Pelleas Street	\$1,700,000	\$4,800,000	\$3,100,000
96 Glover Street	\$3,390,000	\$5,383,333	\$1,993,333
13 Lancaster Street	\$1,900,000	\$1,278,333	\$621,667

17 Morella Road, Mosman: the \$23 million luxury sale exceeds the highest training price (\$11.55 million). Neighbour averaging cannot reach it. Harbour views, architectural design and bay access are missing. The retained sold-card bathroom count is three; a current profile says four. [4]

10 Cyprian Street, Mosman: views, a pool and rebuilding potential are absent. Its encoded inputs match Morella exactly, including March 2026, so both receive \$5.391 million despite very different outcomes. [5]

53-53A Pelleas Street, Blacktown: seven bedrooms and four bathrooms attract inappropriate neighbours: two Mosman houses at \$9 million and \$3.9 million, alongside a \$1.5 million local house. The model misses the local context of this dual-living home. [6]

96 Glover Street, Mosman: the listing reports a 379 m² block, shared driveway and \$3,600 quarterly strata charges. These omitted details could help explain the overestimate. One neighbour costs \$8.7 million. This is a plausible explanation, not a proven causal effect. [7]

13 Lancaster Street, Blacktown: a recent custom-built, multigenerational house is compared with three local sales between \$1.235 million and \$1.34 million. Construction age and finish quality are missing, so its premium is underestimated. [8]

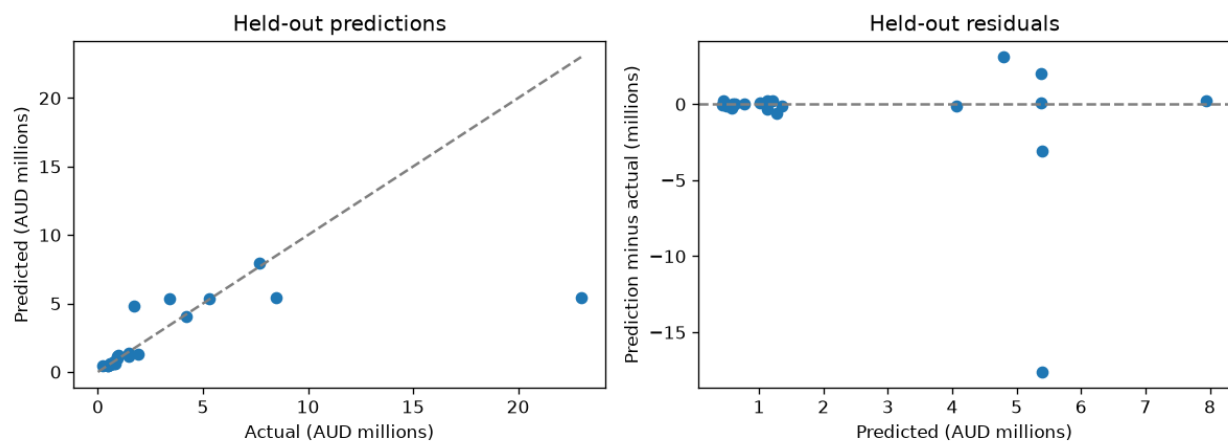


Figure 4. Held-out predictions and errors. Rare, expensive sales dominate squared error.

5. ML, LLM and illustrative estimates

Ten held-out properties received the same six raw inputs for ML and Codex estimates. LLM answers were frozen before these prices were revealed. Codex had seen earlier market examples, so this was not a fresh-context or equal-training-information experiment. The prompt and protocol are included. Genuine student estimates are missing; the requested invented column was added after outcomes were known.

ID / suburb	Actual	ML	LLM	AI illustration*
C01 / Parramatta	\$250,000	\$443,000	\$310,000	\$300,000
C02 / Parramatta	\$685,000	\$572,333	\$810,000	\$750,000
C03 / Blacktown	\$465,000	\$438,000	\$450,000	\$450,000
C04 / Blacktown	\$900,000	\$1,131,000	\$1,150,000	\$1,100,000
C05 / Blacktown	\$1,700,000	\$4,800,000	\$1,750,000	\$1,500,000
C06 / Blacktown	\$590,000	\$499,667	\$520,000	\$550,000
C07 / Mosman	\$3,390,000	\$5,383,333	\$4,800,000	\$5,000,000
C08 / Mosman	\$5,300,000	\$5,383,333	\$4,700,000	\$5,000,000
C09 / Mosman	\$4,200,000	\$4,070,000	\$5,100,000	\$5,000,000
C10 / Mosman	\$23,000,000	\$5,391,000	\$4,800,000	\$5,000,000

Table 3. *Post-hoc demonstration values, NOT personal estimates or blind predictions. Addresses and inputs are mapped to these IDs in the notebook and CSV files.

Approach	MAE	RMSE	R ²
ML	\$2,356,967	\$5,690,310	0.242
LLM	\$2,168,000	\$5,783,489	0.217

The LLM has lower MAE, but ML has lower RMSE. Both badly miss the \$23 million sale. The LLM is closer for the seven-bedroom Blacktown home; ML is closer for the \$5.3 million Mosman home. Neither knows omitted exceptional features. Ten selected cases cannot establish a general winner.

Human judgement could add inspection knowledge, condition and local comparables, but that benefit is not measured here. A valid human comparison now requires new blinded cases. Invented estimates cannot establish whether a person outperforms either model.

Revisiting the feature expectations

Validation permutation importance ranks suburb first, then month and bathrooms. The initial guess is partly supported: location leads, but type and bedrooms rank lower. Importance is not causal; correlated features share information, and month may identify the collection pattern rather than price growth.

6. Deployment and use

The fitted 95-row KNN pipeline is saved with joblib. A Flask endpoint uses that pipeline; a static HTML/JavaScript interface loads exported scaling, imputation, categories and neighbours. Python and browser predictions match for all 119 records. Equal-distance ties use a stable row order in both implementations. This consistency fix followed initial evaluation and did not tune test performance.

The screenshot shows a web application titled "Sydney housing price predictor" with the subtitle "Enter a property's details to estimate its sale price in Australian dollars." The page is divided into two main sections: "Property details" and "Estimated sale price".

Property details

Suburb: Parramatta (dropdown menu)
Property type: Apartment (dropdown menu)
Bedrooms: 2 (text input)
Bathrooms: 2 (text input)
Parking spaces: 1 (text input)
Valuation date: 08/01/2026 (calendar icon)
A "Predict price" button is located at the bottom of the form.

Estimated sale price

Enter the details

Select Predict price to see the model's estimate.

About this estimate

The model averages the prices of 3 similar training examples. Similarity includes suburb, type, rooms, parking and sale month.

It was trained on 95 properties and tested on 24. Its average absolute test error was about **\$1.20 million**, with a few very large errors.

Study prototype only. It can miss views, condition, strata costs and unusual luxury features. This is not a professional valuation.

Data: 119 disclosed sales in Parramatta, Blacktown and Mosman. Sources: Domain and realestate.com.au. This small sample does not represent the whole Sydney market.

Figure 5. Actual local browser screenshot: choose features before prediction.

The screenshot shows the same web application as Figure 5, but now the "Estimated sale price" section displays the result of the prediction.

Estimated sale price

\$692,333

Parramatta - Apartment - 2 bedrooms

About this estimate

The model averages the prices of 3 similar training examples. Similarity includes suburb, type, rooms, parking and sale month.

It was trained on 95 properties and tested on 24. Its average absolute test error was about **\$1.20 million**, with a few very large errors.

Study prototype only. It can miss views, condition, strata costs and unusual luxury features. This is not a professional valuation.

Data: 119 disclosed sales in Parramatta, Blacktown and Mosman. Sources: Domain and realestate.com.au. This small sample does not represent the whole Sydney market.

Figure 6. Actual local browser screenshot: Parramatta apartment, 2 bedrooms, 2 bathrooms, 1 parking, 1 August 2026 gives \$692,333.

Choose suburb and type, enter room counts, parking and date, then click Predict price. Zero bedrooms means studio; blank parking means unknown. The app shows the estimate and study limitations. Dates beyond training coverage trigger a warning. The hosted version is owner-private; tutor access must be arranged separately.

Reproduction, reflection and sources

Build and run

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python train.py
python export_model.py
python verify_project.py
python build_report.py
python app.py
```

Use the extracted project folder. On Windows activate with `.venv\Scripts\activate`. Open `http://127.0.0.1:5000`. For notebook outputs, select this environment in Jupyter/VS Code and run all cells. The static folder `site/dist` can also be hosted directly. Verification needs Node.js. Full setup and archive instructions are in `README.md`.

Reflection

My main takeaway is that better data matters more than a more complicated model. I expected random forest to perform best, but KNN had the lowest cross-validation MAE. Its large test errors show why I should not rely on one score. Missing area, views, condition and ownership costs limit what the model can learn. I would collect more consistent property details and use building-grouped and time-based tests before trusting it on new sales.

I would keep the app simple so its inputs and limitations are easy to understand. Deploying it also shows why preprocessing must match training exactly. More computing power alone would not fix missing information. Suburb can reflect historic inequality, and selective listings can favour some housing types. I would compare errors across groups and avoid using this prototype for lending or decisions about access to housing.

References (accessed 16–17 September 2026)

1. SIT720 supplied task sheet and all Week 8/9 note screenshots
2. Domain sold listings: Parramatta and Blacktown; exact page URLs in [housing_sales.csv](#)
3. realestate.com.au sold listings: Mosman; exact page URLs in [housing_sales.csv](#)
4. Morella Road sold listing
- 4a. Morella Road current profile (bathroom discrepancy)
5. Cyprian Street sold listing
6. Pelleas Street sold listing
7. Glover Street sold listing
8. Lancaster Street sold listing
9. scikit-learn: cross-validation
10. scikit-learn: leakage and common pitfalls
11. scikit-learn: ensemble methods

Appendix: core code

Core steps adapted from train.py. The executed notebook and all source files in the archive contain the complete workflow, outputs, plots and application. See stable_knn.py for deterministic tie handling.

```
def make_features(frame):
    features = frame[["suburb", "property_type", "bedrooms",
                     "bathrooms", "parking"]].copy()
    dates = pd.to_datetime(frame["sale_date"])
    features["sale_month"] = (dates.dt.year - 2025) * 12 + dates.dt.month - 1
    features["bathrooms_per_bedroom"] = (
        features["bathrooms"] / features["bedrooms"].clip(lower=1))
    return features

numeric = ["bedrooms", "bathrooms", "parking", "sale_month",
           "bathrooms_per_bedroom"]
categorical = ["suburb", "property_type"]
preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                     ("scale", StandardScaler())]), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore", sparse_output=False),
     categorical)
])
cv = KFold(n_splits=5, shuffle=True, random_state=42)

# Example: KNN selection. The notebook repeats this for all models.
pipe = Pipeline([("preprocess", preprocess),
                 ("model", StableKNNRegressor())])
search = GridSearchCV(pipe, {"model__n_neighbors": [3, 5, 9]},
                      scoring={"mae": "neg_mean_absolute_error",
                              "rmse": "neg_root_mean_squared_error", "r2": "r2"},
                      refit="mae", cv=cv, return_train_score=True)
search.fit(X_train, y_train)

predictions = best_model.predict(X_test)
mae = mean_absolute_error(y_test, predictions)
rmse = np.sqrt(mean_squared_error(y_test, predictions))
r2 = r2_score(y_test, predictions)
joblib.dump(best_model, ROOT / "model/housing_model.joblib")
```

The appendix abbreviates variable setup and the model loop; run the notebook or train.py for a complete runnable program. The saved pipeline includes preprocessing, preventing separate application transformations from drifting away from training.

Application structure

File	Responsibility
app.py	Serve the page and POST /api/predict
prediction.py	Validate input, engineer features and call Python pipeline
export_model.py	Export fitted KNN preprocessing and neighbour data
site/dist/predict.js	Apply the same transformations and stable KNN average
site/dist/app.js	Read the form, validate and display result
verify_project.py	Check data, held-out split, API and Python/JavaScript parity